PRACTICAL NUMBER: 5

AIM: Maintain employee database and implement the following task:

i. Display the names of employee based on salary

ii. Display the name of employee having maximum and minimum Salary

iii. Find the total monthly expenses of the company on employee salary

***PROGRAM*:**

```
//sheikh wasimuddin cse(iot) 60  23071044 //
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct EmployeeNode {
   char name[50];
   float salary;
   struct EmployeeNode *left;
   struct EmployeeNode *right;
} EmployeeNode;
EmployeeNode* createNode(char* name, float salary) {
   EmployeeNode* newNode =
(EmployeeNode*)malloc(sizeof(EmployeeNode));
   strcpy(newNode->name, name);
```

```
    newNode->salary = salary;
    newNode->left = newNode->right = NULL;
    return newNode;
}
EmployeeNode* insert(EmployeeNode* root, char* name, float
salary) {
    if (root == NULL) {
        return createNode(name, salary);
    }
    if (salary < root->salary) {
        root->left = insert(root->left, name, salary);
    } else {
        root->right = insert(root->right, name, salary);
    }
    return root;
}
void inorderTraversal(EmployeeNode* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
        printf("Name: %s, Salary: %.2f\n", root->name, root->salary);
        inorderTraversal(root->right);
    }
}
```

```
EmployeeNode* findMaxSalary(EmployeeNode* root) {
    if (root == NULL) return NULL;
    while (root->right != NULL) {
        root = root->right;
    }
    return root;
}
EmployeeNode* findMinSalary(EmployeeNode* root) {
    if (root == NULL) return NULL;
    while (root->left != NULL) {
        root = root->left;
    }
    return root;
}
float calculateTotalExpenses(EmployeeNode* root) {
    if (root == NULL) return 0;
    return root->salary + calculateTotalExpenses(root->left) +
calculateTotalExpenses(root->right);
}
int main() {
    EmployeeNode* root = NULL;
    root = insert(root, "sheikh", 50000);
    root = insert(root, "tanmay", 75000);
    root = insert(root, "parth", 20000);
```

```
    root = insert(root, "tejas", 40000);
    root = insert(root, "omkar", 30000);
    root = insert(root, "vedant", 49000);

    printf("Employees sorted by salary:\n");
    inorderTraversal(root);
    EmployeeNode* maxSalaryNode = findMaxSalary(root);
    if (maxSalaryNode != NULL) {
        printf("\nEmployee with maximum salary: %s, Salary: %.2f\n",
maxSalaryNode->name, maxSalaryNode->salary);
    }
    EmployeeNode* minSalaryNode = findMinSalary(root);
    if (minSalaryNode != NULL) {
        printf("\nEmployee with minimum salary: %s, Salary: %.2f\n",
minSalaryNode->name, minSalaryNode->salary);
    }
    float totalExpenses = calculateTotalExpenses(root);
    printf("\nTotal monthly expenses: %.2f\n", totalExpenses);
    return 0;
}
```

OUTPUT:

main.c

```c
//sheikh wasimuddin cse(iot) 60  23071044 //
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct EmployeeNode {
    char name[50];
    float salary;
    struct EmployeeNode *left;
    struct EmployeeNode *right;
} EmployeeNode;

EmployeeNode* createNode(char* name, float salary) {
    EmployeeNode* newNode = (EmployeeNode*)malloc(sizeof(EmployeeNode));
    strcpy(newNode->name, name);
    newNode->salary = salary;
    newNode->left = newNode->right = NULL;
    return newNode;
}

EmployeeNode* insert(EmployeeNode* root, char* name, float salary) {
    if (root == NULL) {
        return createNode(name, salary);
    }
    if (salary < root->salary) {
        root->left = insert(root->left, name, salary);
    } else {
        root->right = insert(root->right, name, salary);
    }
    return root;
}

void inorderTraversal(EmployeeNode* root) {
    if (root != NULL) {
        inorderTraversal(root->left);
```

```
Employees sorted by salary:
Name: parth, Salary: 20000.00
Name: omkar, Salary: 30000.00
Name: tejas, Salary: 40000.00
Name: vedant, Salary: 49000.00
Name: sheikh, Salary: 50000.00
Name: tanmay, Salary: 75000.00

Employee with maximum salary: tanmay, Salary: 75000.00

Employee with minimum salary: parth, Salary: 20000.00

Total monthly expenses: 264000.00


...Program finished with exit code 0
Press ENTER to exit console.
```

CONCLUSION: This practical was performed successfully.